AUTONOMOUS CI/CD INCIDENT TRIAGER & SELF-HEALER

Presented By :

Om Kalpesh Kshirsagar (23BAI1351)
Adwait Vikram Pande (23BAI1263)
Suryansh Rai (23BAI1469)

Guided By:

Dr. Raja M







Current Situation & Domain Knowledge

Cloud-native systems continuously generate incidents across multiple sources: application logs, infrastructure logs, monitoring alerts, CI/CD pipelines, deployments, Git commits, configuration changes, and service health endpoints. Manual correlation across these fragmented sources is slow and error-prone, creating critical delays during production incidents.

Incident Lifecycle: An alert is triggered by a threshold breach or anomaly. Engineers begin investigation by manually querying logs, dashboards, recent deployments, and configuration changes. Connecting events across sources requires significant domain expertise and time under pressure.

Recovery follows root cause identification, requiring rollback, restart, or configuration fix, then verification that the system has stabilized. Each phase demands cross-source reasoning: a single incident may span Git history, a failed CI/CD stage, a misconfigured deployment, and cascading service failures across distributed components.

Problem Statement

The Core Challenge

Engineers under time pressure must manually inspect logs, metrics, CI/CD pipelines, Git history, deployments, and config, simultaneously, to diagnose a single incident.

What failed, and when did it start?

What changed just before the failure?

What recovery action is safe to execute?

Fragmented Evidence

Evidence is scattered across disconnected sources with no unified timeline, forcing engineers to mentally correlate events while the system remains degraded.

Which component is responsible?

What evidence supports each hypothesis?

How do we verify recovery without causing further harm?

Literature Review

Paper 1: Multi-Agent SRE & Self-Healing

Connects Git, build, deployment, config, logs, metrics and health into unified evidence.

Hypothesis-based RCA with safety-aware remediation and closed-loop verification.

Papers 1 – 3

Paper 2: LLM-based Agents for RCA
Validates LLM agents for root cause analysis in cloud-native environments.

Extends to full CI/CD lifecycle correlation beyond single-source analysis.

Integrates RCA with controlled remediation and closed-loop verification.

Paper 3: LogRCA : Logs as Major RCA Evidence
Logs contain large irrelevant volumes; filtering relevant windows is essential.

Literature Review

RCA Surveys, Causal Inference & Benchmarks

Paper 4 : Comprehensive Survey on RCA in (Micro)Services
Organizes RCA around metrics, traces, logs, multimodal data, and LLMs. RCA process: collection → analysis → hypothesis formulation → validation. Highlights service dependencies, fault propagation, and noisy heterogeneous data. Supports multi-source correlation as conceptual foundation for project RCA.

735 failure cases across 3 microservice systems and 11 fault types.

Combines metrics, logs, and traces; includes code-level faults and root-cause annotations.

Demonstrates need for selective evidence retrieval; informs controlled incident scenario evaluation.



Papers 4 – 6

Paper 5: Causal Inference RCA for Microservices
Symptom location often differs from the true causal source. Emphasizes dependencies, event ordering, and cause-effect reasoning. Supports hypothesis-based investigation across service boundaries.

No single causal approach works for every failure type.

Motivates testing multiple incident scenarios in evaluation.

Paper 6: RCAEval: Reproducible Benchmark

Literature Review

Paper 7: AIOpsLab

Framework to design, develop and evaluate AI agents for autonomous cloud operations.

Interactive environments matter, mitigation changes system state dynamically.

Motivates controlled experiments for agentic investigation and remediation.

Papers 7–9

Paper 8: Flow-of-Action SOP-enhanced LLM multi-agent RCA; structured procedures constrain tool use and reduce hallucinated actions.

Supports specialized agents plus a coordinator for safe, tool-driven investigation.

Paper 9: Cloud-Based AI Systems combines ML fault detection with LLM semantic interpretation for heterogeneous operational data.

Project extends this with explicit evidence-driven RCA and CI/CD pipeline context.

Literature Review Papers 10–12

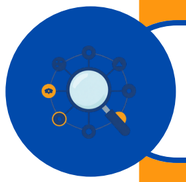
Paper 10: MicroRCA-Agent : multimodal evidence quality, compression and alignment are crucial for LLM-based RCA. Modality-specific preprocessing improves signal quality. Supports evidence collection, context builder, and RCA engine components.

Paper 12: GALA

Project extends MicroRCA-Agent with Git, deployment, and CI/CD context.

Ablation-style evaluation strategy is directly relevant to this project.

GALA uses four phases: hypothesis generation, diagnostic bundles, graph-guided investigation, and final diagnosis with action recommendations.



Paper 11: LATS-RCA

Reflection-guided tree search explores competing root-cause hypotheses without premature commitment.

Multiple specialized log and metric agents coordinated by a supervisor agent.

Directly supports hypothesis generation, ranking, and agent specialization design.

Graph-augmented LLM agents combine service-dependency graphs, metrics, logs, and traces.

Literature Review: Papers 13–14

Paper 13: LLM for Automated RCA

Combines exception aggregation, ML-based classification, and code-aware LLM reasoning for microservice RCA.

Hybrid architecture: ML filters high-volume events; LLM handles contextual reasoning.

Code-aware diagnosis enables precise fault localization at source level.

Informs incident classification and evidence-driven code-level diagnosis layer.

Paper 14: AI-Augmented Self-Healing

Combines health probes, AI diagnosis, and predefined remediation playbooks in a closed-loop self-healing system.

Kubernetes health probes provide objective, real-time service state signals.

Playbooks constrain autonomous actions, ensuring safe and bounded remediation.

Closed-loop: detect → diagnose → remediate → verify; informs Recovery Planner and Post-Remediation Verification.

Literature Synthesis

Prior literature addresses key components of autonomous incident management but no single system integrates all layers into a CI/CD-aware, evidence-driven, closed-loop pipeline.

Existing Work Covers

Multi-agent RCA & self-healing (Papers 1, 11, 14)

Multimodal telemetry, causal & graph reasoning (Papers 4, 5, 6, 12)

SOP constraints, benchmarks & AIOps eval (Papers 7, 8, 13)

Research Gap

No existing work unifies CI/CD-aware evidence collection, timeline correlation, hypothesis-based RCA with confidence tracking, safety-aware remediation, and closed-loop verification in one architecture.

CI/CD-aware evidence collection + timeline correlation across Git, build, deploy, config, logs & metrics

Hypothesis-based RCA with confidence scores, supporting & contradicting evidence tracking

Safety-aware remediation + closed-loop verification the integrated contribution



Proposed Methodology Overview

The methodology follows an agentic investigation loop not one-shot LLM prompting. Each stage feeds context forward, enabling iterative hypothesis testing and evidence-driven decision-making before any remediation action is taken.

Investigation Loop

- Alert → Ingestion → Data Collection → Context Builder → AI Analyzer → RCA
- RCA → Recovery Planner → Safety/Policy Check → Safe Action → Verification
- Verification → Post-Incident Report (or loop back to investigation)

Agentic Investigation & RCA



Investigation Process

The agent identifies missing context and iteratively queries relevant data sources to build a complete incident picture.

Queries logs, metrics, Git history, CI/CD pipelines, deployments, config, and service health APIs.

Constructs a unified event timeline correlating changes and anomalies before and after failure.

Determines information gaps and re-queries tools until sufficient evidence is collected.



Hypothesis & Diagnosis

Multiple hypotheses are generated, tested against evidence, and ranked by confidence to produce an explainable diagnosis.

Generates competing root-cause hypotheses from correlated evidence across all data sources.

Tests each hypothesis against additional evidence; records supporting and contradicting signals.

Ranks causes with confidence scores and produces a structured, explainable RCA diagnosis.

Proposed System Architecture

Pipeline Flow: Incident/Alert → Ingestion Layer → Data Collector → Context Builder → AI/RCA Engine → Recovery Planner → Safety Layer → Action Executor → Verification → Post-Incident Report.

Data Sources feeding the Collector: application logs, infrastructure metrics, Git history, CI/CD pipeline events, deployment records, configuration snapshots, and service health probes.

Feedback Loops: Verification results feed back into the investigation cycle if recovery is incomplete. The Reporting layer captures evidence, decisions, and outcomes for every incident handled by the system.

Module Description I



Incident Ingestion & Data Collector

Incident Ingestion: Receives alerts from monitoring, CI/CD pipelines, and health probes; normalizes events into a unified incident schema with severity, source, and timestamp.

Data Collector gathers multi-source evidence:

Logs, metrics, Git commits, CI/CD run status, deployment history, configuration diffs, and service health signals.

Output: Enriched incident record with normalized, timestamped, multi-source evidence ready for timeline correlation.



Incident Context Builder

Purpose: Assembles all collected evidence into a structured incident context object ready for AI analysis and RCA.

Context outputs include:

Incident summary, correlated timeline, recent change events, filtered evidence windows, and dependency map.

Input: Enriched incident record. Output: Structured context bundle passed to the AI Incident Analyzer for hypothesis generation.

Module Description II

AI RCA & Timeline Correlation

AI Root Cause Analyzer generates multiple hypotheses ranked by confidence score, each with supporting and contradicting evidence.

Hypothesis example: "Deployment v2.3.1 introduced a misconfigured env variable" Confidence: 87% Supporting: config diff, error spike at deploy time Contradicting: health probe passed initially.

Timeline Correlation sequences events across Git commits, CI/CD runs, deployments, config changes, log anomalies and metric deviations to identify causal ordering.

Hypotheses are tested iteratively agent queries additional tools to confirm or refute each candidate cause before ranking the final diagnosis.

Recovery Planner

Maps confirmed root cause to allowed remediation playbooks: rollback, restart, scale, feature flag, or controlled retry.

Recovery recommendation example: Rollback to v2.3.0, Reason: config regression confirmed, Risk: low (stateless service) Expected result: error rate returns below 0.5% within 2 min.

Planner outputs structured recommendation: action type, target component, rationale, risk level, rollback plan, and expected post-action health signals.

All recommendations are passed to the Safety Layer for allowlist validation, dry-run check, and approval gate before any autonomous action is executed.

THANK YOU

